Лекция № 03. Интегрированные среды разработки для создания независимых программ

Интегрированные среды разработки для создания независимых программ

> **Интегрированная среда разработки (ИСР)** – это система программных средств, используемая программистам для разработки программного обеспечения.

В английском языке такая среда называется **Integrated development environment** или сокращённо **IDE**.

ИСР обычно включает в себя т**екстовый редактор, компилятор, интерпретатор, средства автоматизации разработк**и и **сборки программного обеспечени**я и **отладчик**. Иногда также содержит средства для интеграции с **системами управления версиями** и разнообразные инструменты для упрощения конструирования графического интерфейса пользователя. Многие **современные среды разработки** также включают окно **просмотра программных классов**, и**нспектор объектов и диаграмму иерархии классов** – для использования при объектно-ориентированной разработке ПО. Большинство современных ИСР предназначенны для разработки программ на нескольких языках программирования одновременно.

Один из частных случаев ИСР – среды визуальной разработки, которые включают в себя возможность визуального редактирования интерфейса программы.

Основным окном, является текстовый редактор, который используется для ввода исходного кода в ИСР и ориентирован на работу с последовательностью символов в текстовых файлах. Такие редакторы обеспечивают **расширенную функциональность** – подсветку синтаксиса, сортировку строк, шаблоны, конвертацию кодировок, показ кодов символов и т. п. Иногда их называют редакторами кода, так как основное их предназначение – написание исходных кодов компьютерных программ.

> **Подсветка синтаксиса** – выделение синтаксических конструкций текста с использованием различных цветов, шрифтов и начертаний.

Обычно применяется в текстовых редакторах для облегчения чтения исходного текста, улучшения визуального восприятия. Часто применяется при публикации исходных кодов в Интернет.

Одна из наиболее важных частей ИСР – отладчик, который представляет собой модуль среды разработки или отдельное приложение, предназначенное для поиска ошибок в программе. Отладчик позволяет выполнять пошаговую трассировку, отслеживать, устанавливать или изменять значения переменных в процессе выполнения программы, устанавливать и удалять контрольные точки или условия остановки и т.д.

Наиболее распространёнными отладчиками являются:

- GNU Debugger – отладчик программ от проекта GNU;
- IDA – дизассемблер и низкоуровневый отладчик для операционных систем семейства Windows и GNU/Linux;
- Microsoft Visual Studio – среда разработки программного обеспечения, включающая средства отладки от корпорации Microsoft;
- OllyDbg – бесплатный низкоуровневый отладчик для операционных систем семейства Windows;
- SoftICE – низкоуровневый отладчик для операционных систем семейства Windows;
- Dr. Watson – стандартный отладчик Windows, позволяет создавать дампы памяти;
- WinDbg – бесплатный отладчик от корпорации Microsoft.

Основным процессом отладки является трассировка.

> **Трассировка** – это процесс пошагового выполнения программы.

В режиме трассировки программист видит последовательность выполнения команд и значения переменных на данном шаге выполнения программы, что позволяет легче обнаруживать ошибки. Трассировка может быть начата и окончена в любом месте программы, выполнение программы может останавливаться на каждой команде или на точках останова, трассировка может выполнятся с заходом в процедуры/функции и без заходов.

Наиболее важным модулем ИСР при совместной разработке проектов средней и высокой степени сложности является **система управления версиями**. Система управления версиями (английская аббревиатура CVS) - программное обеспечение для облегчения работы с изменяющейся информацией. Она позволяет хранить несколько версий одного и того же документа, при необходимости, возвращаться к более ранним версиям, определять, кто и когда сделал то или иное изменение и многое другое.

Такие системы наиболее широко применяются при разработке программного обеспечения, для хранения исходных кодов разрабатываемой программы. Однако они могут с успехом **применяться и в других областях**, в которых ведётся работа с большим количеством непрерывно изменяющихся **электронных документов**, в частности, они всё чаще применяются в **САПР**, обычно в составе систем управления данными об изделии. Управление версиями используется в инструментах конфигурационного управления различных устройств и систем.

В нашей стране, возможно в связи с малым количеством масштабных проектов, **системы управления версиями распространение не получили**, несмотря на то, что их использование является залогом успешной реализации крупных проектов. В связи с этим остановимся подробнее на этой возможности ИСР.

Большинство систем управления версиями используют централизованную модель, когда имеется единое хранилище документов, управляемое специальным сервером, который и выполняет большую часть функций по управлению версиями. Пользователь, работающий с документами, должен сначала получить нужную ему версию документа из хранилища; обычно создаётся локальная копия документа, так называемая «рабочая копия». Может быть получена последняя версия или любая из предыдущих, выбранная по номеру версии или дате создания, иногда и по другим признакам. После того, как в документ внесены нужные изменения, новая версия помещается в хранилище. В отличие от простого сохранения файла, предыдущая версия не стирается, а тоже остаётся в хранилище и может быть получена оттуда в любое время. Сервер может использовать дельтакомпрессию – способ хранения документов, при котором сохраняются только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных.

Иногда создание новой версии выполняется незаметно для пользователя (прозрачно) – либо с помощью прикладной программы, имеющей встроенную поддержку такой функции, либо за счёт использования специальной файловой системы. В последнем случае пользователь просто работает с файлом как обычно, и при сохранении файла автоматически создаётся новая версия.

Часто бывает, что над одним проектом одновременно работают несколько человек. Если два человека изменяют один и тот же файл, то один из них может случайно отменить изменения, сделанные другим. Системы управления версиями отслеживают такие конфликты и предлагают средства их решения. Большинство систем может автоматически объединить (слить) изменения, сделанные разными разработчиками. Однако такое автоматическое объединение изменений, возможно обычно только для текстовых файлов и то, только при условии, что изменялись разные (непересекающиеся) части этого файла. Такое ограничение связано с тем, что большинство систем управления версиями ориентированы на поддержку процесса разработки программного обеспечения, а исходные коды программ хранятся в текстовых файлах. Если автоматическое объединение выполнить не удалось, система может предложить решить проблему вручную.

Часто выполнить слияние невозможно ни в автоматическом, ни в ручном режиме, например, в случае, если формат файла слишком сложен или вообще неизвестен. Некоторые системы управления версиями дают возможность заблокировать файл в хранилище. Блокировка не позволяет другим пользователям получить рабочую копию или препятствует изменению рабочей копии файла (например, средствами файловой системы) и обеспечивает таким образом исключительный доступ только тому пользователю, который работает с документом.

Другие **возможности системы управления версиями** состоят:

- в создании разных вариантов одного документа-ветки, с общей историей изменений до точки ветвления и с разными – после неё.
- возможности узнать, кто и когда добавил или изменил конкретный набор строк в файле;
- ведении журнала изменений, куда пользователи могут записывать пояснения о том, что и почему они изменили в данной версии;
- контролирует права доступа пользователей, разрешении или запрете чтения или изменения данных в зависимости от того, кто запрашивает это действие.

Отдельным классом являются распределённые системы управления версиями. Такие системы используют распределённую модель вместо традиционной клиент-серверной. Они, в общем случае, не нуждаются в централизованном хранилище: вся история изменения документов хранится на каждом компьютере, в локальном хранилище, и при необходимости отдельные фрагменты истории локального хранилища синхронизируются с аналогичным хранилищем на другом компьютере. В некоторых таких системах локальное хранилище располагается непосредственно в каталогах рабочей копии.

Когда пользователь такой системы выполняет обычные действия, такие, как извлечение определённой версии документа, создание новой версии и тому подобное, он работает со своей локальной копией хранилища. По мере внесения изменений хранилища, принадлежащие разным разработчикам, начинают различаться, и возникает необходимость в их синхронизации. Такая синхронизация может осуществляться с помощью обмена патчами или так называемыми наборами изменений (англ. change sets) между пользователями.

Основное преимущество распределённых систем заключается в их гибкости. Каждый разработчик может вести работу независимо, так, как ему удобно, сохраняя промежуточные варианты документов и передавая результаты другим участникам, когда посчитает нужным. При этом обмен наборами изменений может осуществляться по различным схемам. В небольших коллективах участники работы могут обмениваться изменениями по принципу «каждый с каждым», за счет чего отпадает необходимость в создании выделенного сервера. Крупное сообщество, наоборот, может использовать централизованный сервер, с которым синхронизируются копии всех его участников. Возможны и более сложные варианты, например, с созданием групп для работы по отдельным направлениям внутри более крупного проекта.

Для использования систем управления версиями необходимо владеть терминологией этих систем. Общепринятой терминологии не существует, в разных системах могут использоваться различные названия для одних и тех же действий.

Ниже приведены некоторые, наиболее часто используемые варианты. В связи с тем, что системы разрабатывались англоязычным сообществом, а русскоязычная терминология ещё на выработана, используются английские термины.

- **branch (ветвь)** – направление разработки, независимое от других. Ветвь представляет собой копию части (как правило, одного каталога) хранилища, в которую можно вносить свои изменения, не влияющие на другие ветви. Документы в разных ветвях имеют одинаковую историю до точки ветвления и разные – после неё.
- **check-in, commit, submit** – создание новой версии, публикация изменений. Распространение изменений, сделанных в рабочей копии, на хранилище документов. При этом в хранилище создаётся новая версия изменённых документов.
- **Check-out, clone** – извлечение документа из хранилища и создание рабочей копии.
- **Conflict** – конфликтная ситуация, когда несколько пользователей сделали изменения одного и того же участка документа. Конфликт обнаруживается в случае, когда один пользователь уже опубликовал свои изменения, а второй только пытается их опубликовать и система сама не может корректно слить конфликтующие изменения. Поскольку программа может быть недостаточно разумна для того, чтобы определить, какое изменение является «корректным», второму пользователю нужно самому разрешить конфликт (resolve).
- **Merge, integration (слияние)** - объединение независимых изменений в единую версию документа. Осуществляется, когда два человека изменили один и тот же файл или при переносе изменений из одной ветки в другую.
- **Repository (хранилище документов)** - место, где система управления версиями хранит все документы вместе с историей их изменения и другой служебной информацией.
- **Revision (версия документа).** Системы управления версиями различают версии по номерам, которые назначаются автоматически.
- **Tag, label (метка)** – которую можно присвоить определённой версии документа. Метка представляет собой символическое имя для группы документов, причём описывает она не только набор имён файлов, но и ревизию каждого файла. Ревизии включённых в метку документов могут принадлежать разным моментам времени.
- **Trunk, mainline (ствол)** – основная ветвь разработки проекта. Политика работы со стволом может отличаться от проекта к проекту, но в целом она такова: большинство изменений вносится в ствол; если требуется серьёзное изменение, способное привести к нестабильности, создаётся ветвь, которая сливается со стволом, когда нововведение будет в достаточной мере испытано; перед выпуском очередной версии создаётся «релизная» ветвь, в которую вносятся только исправления.
- **Update, sync (обновление, синхронизация)** – синхронизация рабочей копии до некоторого заданного состояния хранилища. Чаще всего это действие означает обновление рабочей копии до самого свежего состояния хранилища. Однако при необходимости можно синхронизировать рабочую копию и к более старому состоянию, чем текущее.
- **Working copy (рабочая копия)** – рабочая (локальная) копия документов.

Рассмотрим возможности ИСР на примере наиболее доступных и популярных версий.



Eclipse (от англ. затмение) – свободная интегрированная среда разработки модульных кроссплатформенных приложений. Развивается и поддерживается некоммерческой организацией Eclipse Foundation (http://www.eclipse.org/).

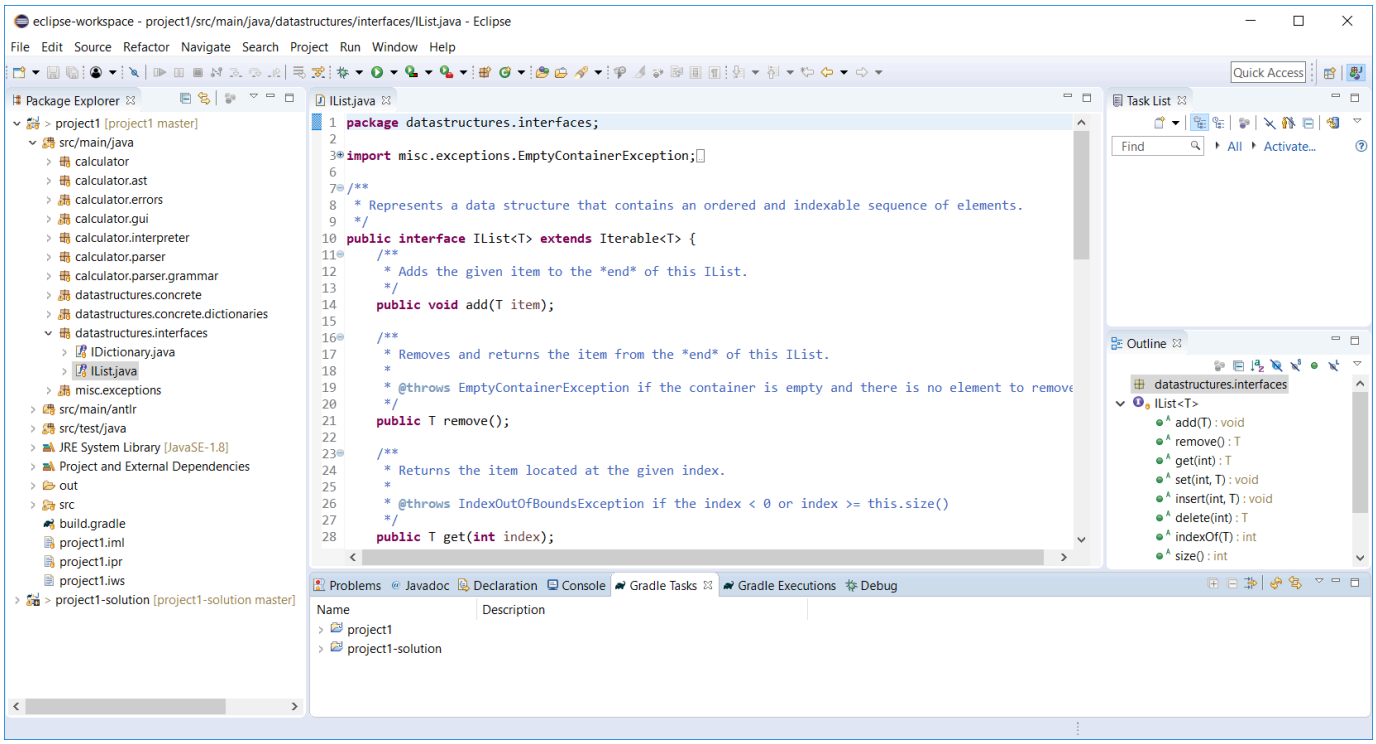
Первоначально Eclipse разрабатывалась фирмой «IBM» в качестве корпоративного стандарта ИСР для разработки на разных языках под платформы от данной компании. По сведениям «IBM», проектирование и разработка стоили 40 млн. долл. Исходный код был полностью открыт и сделан доступным после того, как

Eclipse был передан для дальнейшего развития независимому от «IBM» сообществу.

В основе Эклипс лежат фреймворк OSGi и SWT/JFace, на основе которых разработан следующий слой – RCP (Rich Client Platform, платформа для разработки полноценных клиентских приложений). RCP служит основой не только для Эклипс, но и для других RCP-приложений, например, Azureus и File Arranger. Следующий слой – сам Эклипс, представляющий собой набор расширений RCP: редакторы, панели, перспективы, модуль CVS и модуль Java Development Tools (JDT).

Эклипс – в первую очередь, полноценная Java ИСР, нацеленная на групповую разработку: поддержка CVS входит в поставку Эклипс, активно развиваются несколько вариантов SVN-модулей, существует поддержка VSS и других. В силу бесплатности и высокого качества, Эклипс во многих организациях является корпоративным стандартом для разработки приложений.

Второе назначение Эклипс – служить платформой для разработки новых расширений, чем он и завоевал популярность: любой разработчик может расширить Эклипс своими модулями. Уже существуют C/C++ Development Tools (CDT), разрабатываемые инженерами QNX совместно с «IBM», и средства для языков COBOL, FORTRAN, PHP и прочие от различных разработчиков. Множество расширений дополняет среду Эклипс менеджерами для работы с базами данных, серверами приложений и др.



Интерфейс главного окна Эклипс

Эклипс написана на Java, потому является платформо-независимым продуктом, за исключением библиотеки SWT, которая разрабатывается для всех распространённых платформ. Библиотека SWT используется вместо стандартной для Java библиотеки Swing. Она полностью опирается на нижележащую платформу (операционную систему), что обеспечивает быстроту и натуральный внешний вид пользовательского интерфейса, но иногда вызывает на разных платформах проблемы совместимости и устойчивости приложений.

Основой Eclipse является платформа расширенного клиента (RCP - от англ. rich client platform). **Её компоненты:**

- ядро платформы (загрузка Eclipse, запуск модулей);
- OSGi (стандартная среда поставки комплектов (англ. bundles));
- SWT (портируемый инструментарий виджетов);
- JFace (файловые буферы, работа с текстом, текстовые редакторы);
- рабочая среда Эклипс (панели, редакторы, проекции, мастеры).

Другой популярной свободной ИСР является **КДевелоп** (http://www.kdevelop.org). **КДевелоп (англ. KDevelop)** - свободная среда разработки программного обеспечения для UNIX-подобных операционных систем. Проект стартовал в 1998 году. КДевелоп распространяется согласно лицензии GNU (General Public License).

KDevelop не включает в свой состав компилятор, вместо этого он использует любой компилятор для создания исполняемого кода. Текущая стабильная версия поддерживает большое количество языков программирования, таких как Ада, Bash, C, C++, Фортран, Java, Pascal, Perl, PHP, Python, Ruby и SQL.

КДевелоп использует встроенный компонент – текстовый редактор – через технологию KParts. Основным редактором является Kate.

Функции КДевелоп:

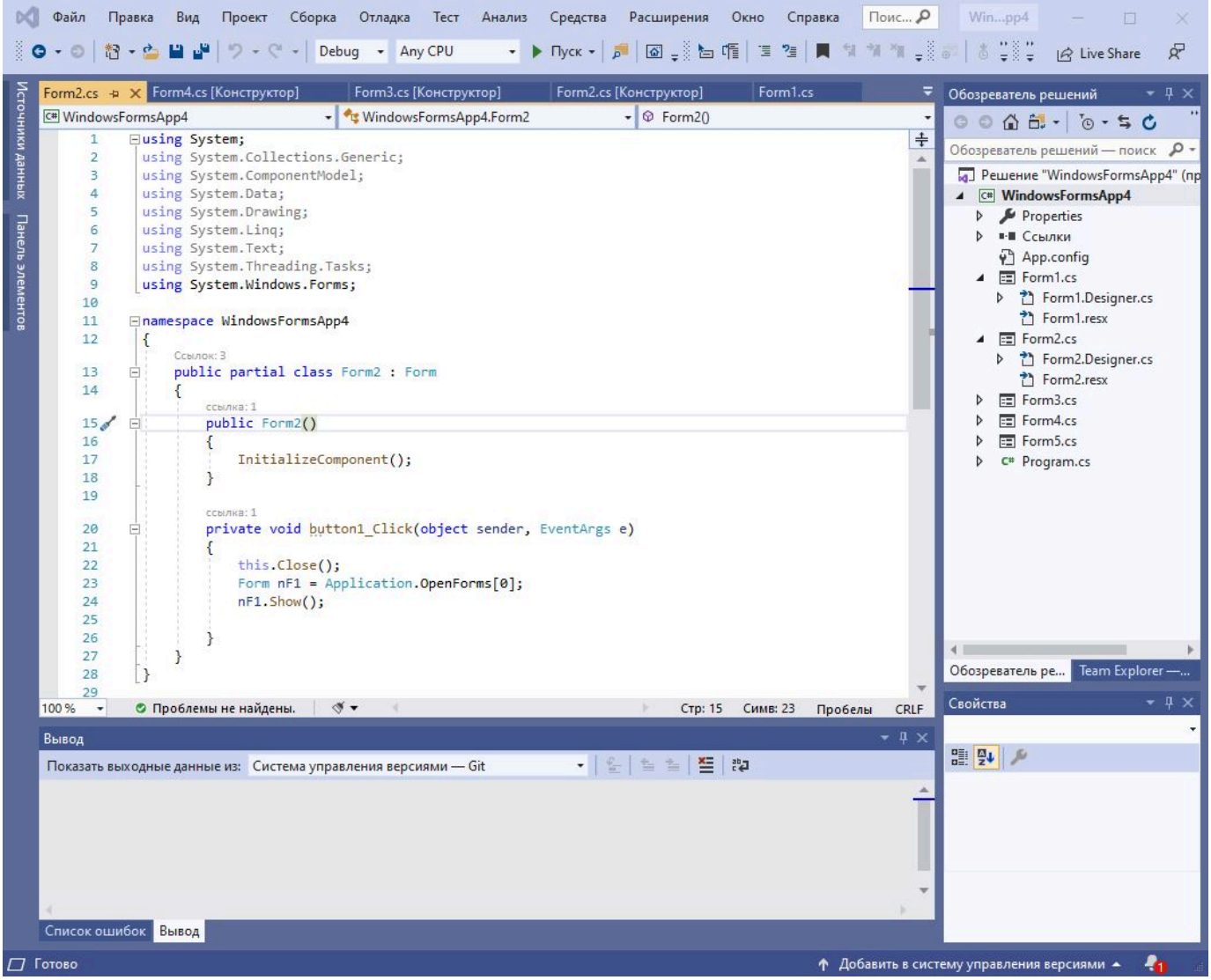
- подсветка исходного кода с учетом синтаксиса используемого языка программирования, который определяется автоматически;
- менеджер проектов для проектов разного типа, таких как Automake, qmake для проектов базирующихся на технологиях Qt и Ant для проектов, базирующихся на Java;
- навигатор классов (Class Browser);
- Front-end для GNU Compiler Collection;
- Front-end для GNU Debugger;
- помощников для генерации и обновления определения классов и платформы (framework);
- автоматическая система завершения кода (Си/C++);
- встроенная поддержка системы документирования исходных кодов (Doxygen);
- одна из систем контроля версий: SCM, CVS, Subversion, Perforce и ClearCase;
- функция Quick Open позволяющая быстро перемещаться по файлам.

KDevelop представляет собой **«подключаемую» архитектуру**. Когда разработчик делает изменения, он должен лишь скомпилировать плагин. Предусмотрена возможность сохранения профилей, указывающих какие плагины должны быть загружены. KDevelop не поставляется со встроенным текстовым редактором, он подключается как плагин. KDevelop не зависит от языка программирования и от платформы, на которой он запускается, поддерживая KDE, GNOME и много других технологий (например, Qt, GTK+ и wxWidgets).

Встроенный отладчик KDevelop позволяет работать графически со всеми средствами отладки, такими как точки останова и трассировки. Он также может работать с динамически подгружаемыми плагинами, в отличие от консольного gdb.

На данный момент существует примерно от 50 до 100 плагинов для данной IDE. Среди наиболее полезных – persistent project-wide code bookmarks, Code abbreviations, позволяющие быстро разворачивать текст, Source formatter, который переформатирует текст для style guide до сохранения, поиск по регулярным выражениям и project-wide поиск/замена.

Последней рассматриваемой ИСР является **Microsoft Visual Studio**. По сути, Microsoft Visual Studio является линейкой продуктов компании «Майкрософт», включающих интегрированную среду разработки программного обеспечения и ряд других инструментальных средств.



Интерфейс Microsoft Visual Studio

Microsoft Visual Studio включает один или несколько компонентов из следующих: Visual Basic.NET, Visual C++, Visual C#, Visual F#, Microsoft SQL Server, Visual InterDev, Visual J++, Visual J#, Visual FoxPro, Visual Source Safe.

Одним из главных преимуществ **Microsoft Visual Studio является высокое качество документирования процесса разработки** и описания возможных проблем в MSDN Library. Однако наиболее интересная для профессионала часть, посвящённая тонкостям разработки, существует только на английском языке. Также компания «Майкрософт» предлагает **бесплатный аналог продукта Visual Studio Community.**

Особенности объектно-ориентированных и структурных языков программирования

Структурированное программирование, как следует из названия, представляет собой метод, который считается предшественником ООП и обычно состоит из хорошо структурированных и разделенных модулей. В этом программировании пользователь может создавать свои собственные пользовательские функции, а также эта методология пытается решить проблемы, связанные с безусловными передачами, чтобы позволить программистам следовать логике программ. Это также требует большей дисциплины на этапе проектирования и логического структурирования.

Пример: Pascal, ALGOL, C, Modula-2 и др.

Объектно-ориентированное программирование, как следует из названия, представляет собой другой подход к программированию, который объединяет данные и функции, которые выполняются на них. В основном оно поддерживает инкапсуляцию, абстракцию, наследование, полиморфизм и т.д. Он также включает функцию скрытия данных, поэтому он более безопасен. Эта модель основана на реальных объектах, которые фокусируются на том, кто должен выполнять задачу, а не на том, что делать.

Пример: C#, Java; C++; Smalltalk; Ada; Eiffel; Object Pascal; Delphi; Visual Basic .NET и т.д.

Разница между структурированным программированием и объектно-ориентированным программированием :

Структурированное программирование	Объектно-ориентированное программирование
Это подмножество процедурного программирования.	Оно основано на концепции объектов, содержащих данные и код.
Программы делятся на небольшие программы или функции.	Программы делятся на объекты или сущности.
Все это направлено на облегчение создания программ с читаемым кодом и повторно используемыми компонентами.	Все дело в создании объектов, которые обычно содержат как функции, так и данные.
Его основная цель - улучшить и увеличить качество, ясность и время разработки компьютерной программы.	Его основная цель - улучшить и повысить качество и продуктивность системного анализа и проектирования.
Он просто фокусируется на функциях и процессах, которые обычно работают с данными.	Он просто фокусируется на представлении структуры и поведения информационной системы в виде крошечных модулей, которые обычно объединяют данные и процессы.
Это метод организации, управления и кодирования программ, который может значительно упростить модификацию и понимание.	Это метод, в котором набор объектов может динамически изменяться и может выполняться просто путем взаимодействия и чтения друг с другом.
В этом методы пишутся глобально, а строки кода обрабатываются одна за другой, т.е. Выполняются последовательно.	В этом метод работает динамически, выполняет вызовы в соответствии с потребностями кода в течение определенного времени.
Обычно это следует "нисходящему подходу”.	Обычно это следует ”восходящему подходу".
Оно обеспечивает меньшую гибкость и абстракцию по сравнению с объектно-ориентированным программированием.	Это обеспечивает большую гибкость и абстракцию по сравнению со структурированным программированием.
Модифицировать структурированную программу и повторно использовать код сложнее по сравнению с объектно-ориентированными программами.	Модифицировать объектно-ориентированные программы и повторно использовать код менее сложно по сравнению со структурированными программами.
Это придает большее значение коду.	Это придает большее значение данным.

Разработка сценариев с помощью специализированных языков

Сценарный язык (язык сценариев, жарг. скриптовый язык; англ. scripting language) — высокоуровневый язык сценариев (англ. script) — кратких описаний действий, выполняемых системой. Разница между программами и сценариями довольно размыта. Сценарий — это программа, имеющая дело с готовыми программными компонентами.

Язык сценариев — это язык программирования, который выполняет задачи в специальной среде выполнения с помощью интерпретатора, а не компилятора.

> Согласно Джону Устерхауту, автору языка Tcl, высокоуровневые языки можно разделить на языки **системного программирования** (англ. system programming languages) и **сценарные языки** (англ. scripting languages). Последние он также назвал **склеивающими языками** (англ. glue languages) или **языками системной интеграции** (англ. system integration languages). Сценарии обычно интерпретируются, а не компилируются, хотя сценарные языки программирования один за другим обзаводятся JIT-компиляторами.

В более узком смысле под скриптовым языком может пониматься специализированный язык для расширения возможностей командной оболочки или текстового редактора и средств администрирования операционных систем.

Есть **два основных типа**: **на стороне сервера** и **на стороне клиента**. Они различаются тем, откуда запускается код, что влияет не только на выбранные языки, но и на производительность и их возможности.

Язык сценариев на стороне сервера. Термин «серверный язык сценариев» относится к тем языкам, которые работают с веб-сервера. Поскольку он выполняется с внутренней стороны, скрипт не виден посетителю. Из-за этого это более безопасный подход. Они часто используются для создания динамических веб-сайтов и платформ, обработки пользовательских запросов, генерации и предоставления данных и прочего. Известный пример серверных сценариев — использование PHP в WordPress.

Примеры: PHP, Python, Node.js, Perl и Ruby.

Клиентский язык сценариев. В отличие от вышеупомянутого, языки сценариев на стороне клиента запускаются из браузера пользователя. Обычно это выполняется во внешнем интерфейсе, что делает его видимым для посетителей и делает его менее уязвимым для эксплойтов и утечек. Таким образом, он часто используется для создания пользовательских интерфейсов и более легких функций, подобных этому. Поскольку они работают локально, они обычно обеспечивают лучшую производительность и, следовательно, не нагружают ваш сервер.

Примеры: HTML, CSS, jQuery и JavaScript.

Как и в случае со сценариями в целом, языки сценариев в основном используются для **автоматизации процессов на уровне приложений**. В командной строке или через веб-сервер они могут выполнять программы, запущенные на веб-сервере, и отображать соответствующие результаты для пользователей. Наиболее распространенные приложения для этих языков различаются в зависимости от их типа.

Например, языки сценариев командной строки, такие как **Bush и PowerShell,** используются для сжатия последовательностей команд. Между тем автоматизация процессов на уровне приложений может быть выполнена с помощью **VBA и AppleScript.**

Кроме того, языки сценариев, такие как **PHP, Python, Perl и JavaScript**, используются на уровне веб-сервера для поддержки динамических веб-сайтов и приложений.

Наконец, **инициативы в области исследования данных и статистики** часто используют этот тип языка для расширения своих процессов. В этих случаях наиболее распространены **R и Python**.
